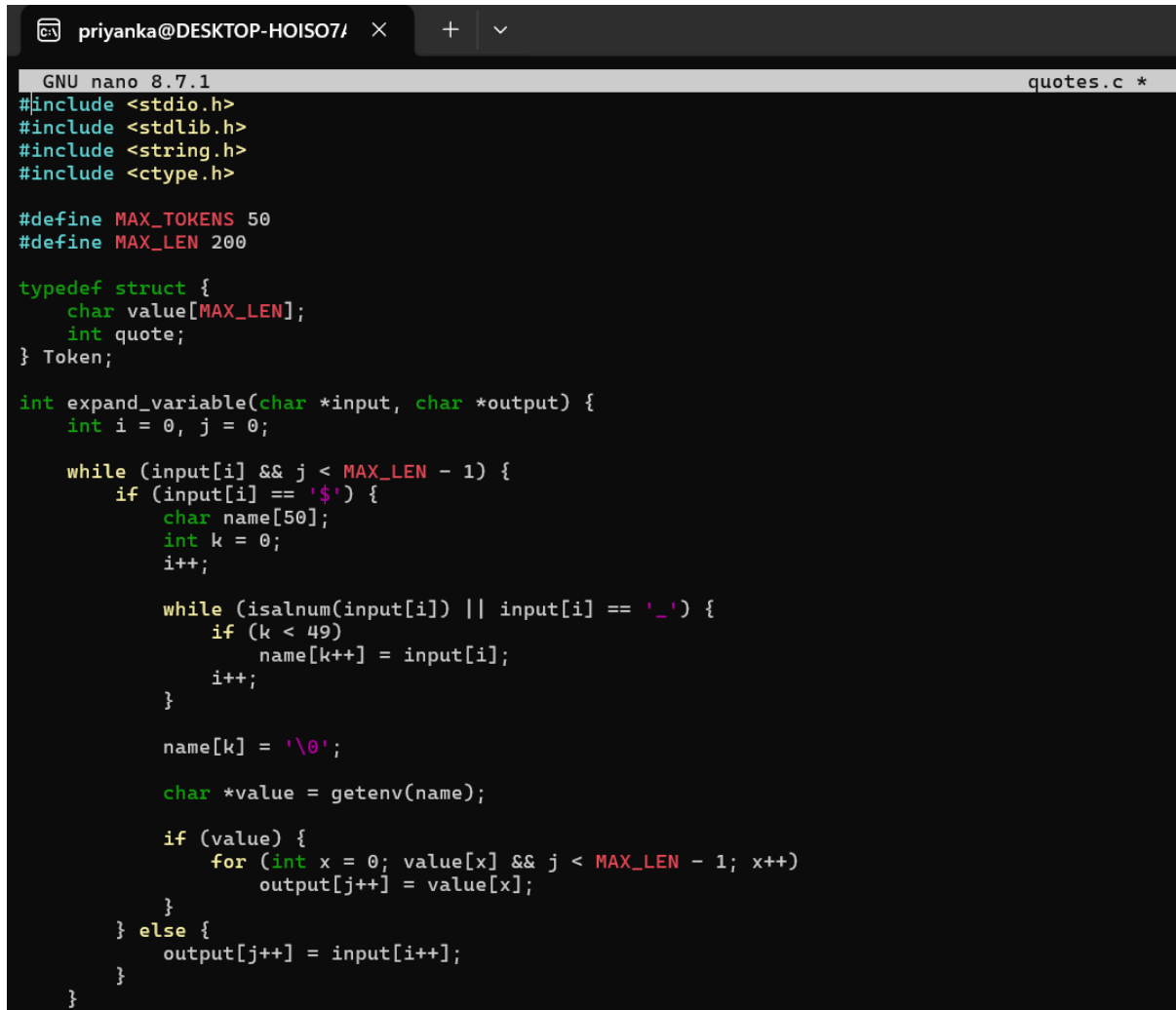


1.To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases

2.To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands

Code:



```
GNU nano 8.7.1 quotes.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_TOKENS 50
#define MAX_LEN 200

typedef struct {
    char value[MAX_LEN];
    int quote;
} Token;

int expand_variable(char *input, char *output) {
    int i = 0, j = 0;

    while (input[i] && j < MAX_LEN - 1) {
        if (input[i] == '$') {
            char name[50];
            int k = 0;
            i++;

            while (isalnum(input[i]) || input[i] == '_') {
                if (k < 49)
                    name[k++] = input[i];
                i++;
            }

            name[k] = '\0';

            char *value = getenv(name);

            if (value) {
                for (int x = 0; value[x] && j < MAX_LEN - 1; x++)
                    output[j++] = value[x];
            }
            else {
                output[j++] = input[i++];
            }
        }
    }
}
```

```
    output[j] = '\\0';
    return j;
}

int tokenize(char *input, Token tokens[]) {
    int count = 0;
    int i = 0;

    while (input[i]) {
        while (isspace(input[i]))
            i++;

        if (!input[i])
            break;

        int quote = 0;
        int j = 0;
        char temp[MAX_LEN];

        if (input[i] == '\\') {
            quote = 1;
            i++;

            while (input[i] && input[i] != '\\') {
                if (j < MAX_LEN - 1)
                    temp[j++] = input[i++];
            }

            if (input[i] != '\\') {
                printf("Error: unmatched single quote\\n");
                return -1;
            }

            i++;
        }
        else if (input[i] == '"') {
            quote = 2;
            i++;

            while (input[i] && input[i] != '"') {
```



priyanka@DESKTOP-HOISO7/



GNU nano 8.7.1

```
        if (j < MAX_LEN - 1)
            temp[j++] = input[i++];
    }

    if (input[i] != '"') {
        printf("Error: unmatched double quote\n");
        return -1;
    }

    i++;
}
else {
    while (input[i] && !isspace(input[i]) &&
           input[i] != '\\' && input[i] != '"') {
        if (j < MAX_LEN - 1)
            temp[j++] = input[i++];
    }
}

temp[j] = '\\0';

if (quote == 2) {
    char expanded[MAX_LEN];
    expand_variable(temp, expanded);
    strcpy(temp, expanded);
}

strcpy(tokens[count].value, temp);
tokens[count].quote = quote;
count++;

if (count >= MAX_TOKENS)
    break;
}

return count;
}

void print_tokens(Token tokens[], int count) {
    printf("\nParsed Tokens:\n");
```



priyanka@DESKTOP-HOISO7/



GNU nano 8.7.1

```
    for (int i = 0; i < count; i++) {
        printf("%d: %s", i + 1, tokens[i].value);

        if (tokens[i].quote == 1)
            printf(" [single quoted]");
        else if (tokens[i].quote == 2)
            printf(" [double quoted]");
        else
            printf(" [unquoted]");

        printf("\n");
    }
}

int main() {
    char input[1000];
    Token tokens[MAX_TOKENS];

    printf("quotes> ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0) {
        printf("Empty command\n");
        return 0;
    }

    int count = tokenize(input, tokens);

    if (count < 0)
        return 1;

    if (count == 0) {
        printf("No tokens found\n");
        return 0;
    }

    print_tokens(tokens, count);
}
```

```
    printf("\nParsing successful\n");  
  
    return 0;  
}  
|
```

Output:

```
priyanka@DESKTOP-H0IS07A:~/process_lab$ cd ~/process_lab  
nano quotes.c  
priyanka@DESKTOP-H0IS07A:~/process_lab$ gcc -Wall -Wextra -std=c11 quotes.c -o quotes  
priyanka@DESKTOP-H0IS07A:~/process_lab$ ./quotes  
quotes> echo 'hello world'  
  
Parsed Tokens:  
1: echo [unquoted]  
2: hello world [single quoted]  
  
Parsing successful  
priyanka@DESKTOP-H0IS07A:~/process_lab$ ./quotes  
quotes> echo "hello beautiful world"  
  
Parsed Tokens:  
1: echo [unquoted]  
2: hello beautiful world [double quoted]  
  
Parsing successful  
priyanka@DESKTOP-H0IS07A:~/process_lab$ echo "hello beautiful world"  
hello beautiful world  
priyanka@DESKTOP-H0IS07A:~/process_lab$ export NAME=Ubuntu  
priyanka@DESKTOP-H0IS07A:~/process_lab$ ./quotes  
quotes> echo '$NAME'  
  
Parsed Tokens:  
1: echo [unquoted]  
2: $NAME [single quoted]  
  
Parsing successful  
priyanka@DESKTOP-H0IS07A:~/process_lab$ ./quotes  
quotes> echo "hello world" 'literal text' test  
  
Parsed Tokens:  
1: echo [unquoted]
```

```
priyanka@DESKTOP-HOIS07A:~/process_lab$ ./quotes
quotes> echo "hello world" 'literal text' test

Parsed Tokens:
1: echo [unquoted]
2: hello world [double quoted]
3: literal text [single quoted]
4: test [unquoted]

Parsing successful
priyanka@DESKTOP-HOIS07A:~/process_lab$ echo "Hello $USER"
Hello priyanka
priyanka@DESKTOP-HOIS07A:~/process_lab$ echo 'hello world'
> |
```